# Plan de pruebas: Filtrar las estaciones por distancia

Aplicación Bicis · Grupo 3 · Historia de usuario 532304 · Autor: Pablo Casal

## Historia de usuario

**Filtrar las estaciones por distancia (ID 532304).** Yo, como visitante temporal, quiero filtrar las estaciones según la distancia a la que se encuentran de manera que solo vea las que puedo alcanzar andando.

**Interfaz esperada:** mock-up de la historia (Docs/Mockups/US532304-FiltrarEstacionesPorDistancia-Mockups.pdf). El diálogo de filtros que se abre con el icono del embudo (el mismo que para bicis y huecos) incluye "Filtrar por distancia": una barra desde 0,1 km hasta el máximo de la ciudad y un campo numérico en km, enlazados entre sí. Con el filtro activo, la lista se ordena de la estación más cercana a la más lejana, con la distancia junto al nombre, y las estaciones cuya distancia no se puede calcular aparecen al final.

## Niveles de prueba

Los niveles de prueba que se van a aplicar son los siguientes:

- **Pruebas de aceptación.** Son los criterios de aceptación acordados con el Product Owner. Se ejecutan de forma manual en el Sprint Review sobre la versión de depuración, con los datos del Anexo 1 y la ubicación del usuario simulada en el centro de Santander, porque las pruebas necesitan una ubicación conocida y una estación sin coordenadas.
- **Pruebas de interfaz.** Un caso de éxito y uno de error, implementados con Espresso sobre la pantalla de estaciones de TUeBICI con los datos del Anexo 1. La ubicación del usuario se simula con un proveedor de ubicación falso inyectado con Hilt, como en las pruebas de interfaz de Detectar la ciudad.
- **Pruebas unitarias.** Se aplican a los métodos nuevos o modificados de StationFilter (modelo) y StationsPresenter, con técnicas de caja negra (partición equivalente y análisis de valores límite), usando JUnit. En el presenter la vista y el repositorio se sustituyen por mocks de Mockito.

A continuación, se muestra una especificación detallada de los casos de prueba a aplicar en cada nivel.

## Criterios de aceptación

Todas las pruebas parten de la pantalla de estaciones de TUeBICI (Santander), que se abre al pulsar TUeBICI en la lista de redes, con la ubicación del usuario en el centro de Santander (Anexo 1). El filtro se abre con el icono del embudo de la barra superior. Las distancias de cada estación al usuario están en la Tabla 10.

### Prueba 1. Caso de éxito

**Contexto:** la red tiene estaciones a distintas distancias del usuario, unas dentro y otras fuera de la distancia que se va a elegir.

1. El usuario selecciona TUeBICI en la lista principal de redes.

2. La aplicación abre la lista de estaciones de esa red (42 estaciones).
3. El usuario abre el filtro, establece una distancia máxima de 1 km y pulsa Aplicar.
4. Se verifica que aparecen las 12 estaciones que están a 1 km o menos, ordenadas de la más cercana ("Ayuntamiento (NE)", a 81 m) a la más lejana ("Tabacalera/Varadero", a 967 m), y al final "Valdenoja 2 (NE)", cuya distancia no se puede calcular ("Santander, ES · 13 estaciones").
5. Se verifica que no aparece ninguna estación que esté a más de 1 km (por ejemplo, "Instituto Pereda (NE)", a 1,06 km).

**Nota:** La distancia se mide en línea recta desde la ubicación actual del usuario hasta cada estación. Según lo acordado con el Product Owner en el Sprint Planning: las estaciones cuya distancia no se puede calcular (porque no informan de sus coordenadas) van al final de la lista; la distancia por defecto es el máximo de la ciudad (la distancia a la estación más lejana), y si no se puede obtener la ubicación del usuario se muestra un mensaje.

**Prueba 2. Estación en el límite**

**Contexto:** alguna estación está exactamente a la distancia máxima que se va a elegir ("Renfe", a 0,437 km del usuario).

1. El usuario selecciona TUeBICI en la lista principal de redes.
2. La aplicación abre la lista de estaciones de esa red.
3. El usuario establece una distancia máxima de 0,437 km (la distancia a "Renfe", redondeada al metro) y aplica el filtro.
4. Se verifica que la estación que está justo en el límite, "Renfe", aparece en la lista: es la última de las 4 estaciones que están dentro, antes de "Valdenoja 2 (NE)".

**Prueba 3. Ninguna estación dentro de la distancia**

**Contexto:** todas las estaciones de la red están más lejos que la distancia que se va a elegir (la más cercana, "Ayuntamiento (NE)", está a 81 m).

1. El usuario selecciona TUeBICI en la lista principal de redes.
2. La aplicación abre la lista de estaciones de esa red.
3. El usuario establece una distancia máxima de 0,05 km (50 m) y aplica el filtro.
4. Se verifica que no aparece ninguna estación en la lista (tampoco "Valdenoja 2 (NE)").
5. Se verifica que aparece el mensaje "No hay estaciones dentro de la distancia seleccionada".
6. El usuario vuelve a abrir el filtro, cambia la distancia a 1 km y aplica el filtro. Se verifica que el mensaje desaparece y se muestran las 13 estaciones de la Prueba 1.

**Prueba 4. Cambio de distancia**

**Contexto:** el usuario ya ha aplicado un filtro por distancia sobre la lista de estaciones.

1. El usuario selecciona TUeBICI en la lista principal de redes.
2. La aplicación abre la lista de estaciones de esa red.
3. El usuario establece una distancia máxima de 1 km y aplica el filtro. Se muestran 13 estaciones.
4. El usuario cambia la distancia máxima a 3 km y vuelve a aplicar el filtro.
5. Se verifica que aparecen las 32 estaciones que están a 3 km o menos, más "Valdenoja 2 (NE)" al final (33), incluidas las que antes quedaban fuera, como "Instituto Pereda (NE)" (1,06 km) y "Magdalena" (2,84 km).
6. Se verifica que no aparece ninguna estación a más de 3 km (por ejemplo, "Alisal (NE)", a 3,40 km).

## Pruebas de interfaz

Se implementan con Espresso en el directorio androidTest, en el paquete es.unican.bicis.activities.stations, que es el de StationsView, la vista que lanza los escenarios. Cada clase se nombra con el criterio de aceptación que implementa (<CriterioAceptación>UITest). Abren StationsView con la red TUeBICI y los datos del Anexo 1, con la ubicación del usuario simulada. Hay un caso de éxito y uno de error.

**Tabla 1. Pruebas de interfaz**

| ID | Criterio | Implementación y entrada | Resultado esperado |
|---|---|---|---|
| UI.1 | Prueba 1. Caso de éxito | FiltrarEstacionesPorDistanciaExitoUITest. Distancia máxima 1 km | 13 estaciones ("Santander, ES · 13 estaciones"), la primera "Ayuntamiento (NE)" y la última "Valdenoja 2 (NE)"; no aparece "Instituto Pereda (NE)" |
| UI.2 | Prueba 3. Ninguna estación dentro de la distancia | FiltrarEstacionesPorDistanciaNingunaDentroUITest. Distancia máxima 0,05 km | Mensaje "No hay estaciones dentro de la distancia seleccionada" en lugar de la lista |

Las pruebas 2 y 4 quedan cubiertas por las pruebas unitarias del modelo (Tabla 3) y del presenter (Tabla 6).

## Métodos a probar

Métodos nuevos o modificados para esta historia de usuario. Los nombres del presenter y de la vista son los del contrato de la historia (IStationsContract, tarea 978170).

**Tabla 2. Métodos a probar**

| Método | Qué hace | Pruebas |
|---|---|---|
| **StationFilter (modelo) filterByMaxDistance(stations, userLat, userLon, maxDistKm)** | Devuelve las estaciones con coordenadas válidas que están a maxDistKm o menos del usuario, en el mismo orden, y detrás las que no tienen coordenadas válidas (solo si alguna estación está dentro). Con la lista null o una distancia menor o igual que 0 devuelve una lista vacía. | Unitarias (Tabla 3) e interfaz (UI.1, UI.2) |
| **StationFilter (modelo) hasValidCoordinates(station)** | Indica si una estación tiene latitud y longitud no null y dentro de rango ([-90, 90] y [-180, 180]). | Unitarias (Tabla 4) |
| **StationFilter (modelo) distanceKm(lat1, lon1, lat2, lon2)** | Calcula la distancia en línea recta, en km, entre dos puntos (fórmula de Haversine). | Unitarias (Tabla 5) |

| Método | Qué hace | Pruebas |
|---|---|---|
| **StationsPresenter onDistanceFilterApplied(maxDistKmText)** | Se llama al aplicar el filtro de distancia. Valida el valor (número mayor que 0, con punto o coma decimal), pide la ubicación a la vista, filtra combinando con el resto de filtros activos, ordena de la estación más cercana a la más lejana (las que no tienen distancia, al final) y pide a la vista la lista con las distancias, el mensaje sin resultados o el error que corresponda. | Unitarias (Tabla 6) e interfaz (UI.1, UI.2) |
| **StationsPresenter getMaxCityDistanceKm()** | Devuelve la distancia del usuario a la estación más lejana con coordenadas válidas. Es el máximo de la barra y el valor por defecto del filtro. Devuelve 0 si no hay ubicación. | Unitarias (Tabla 6) |
| **StationsView getUserLocation() showStationsWithDistance(stations, distances) showNoStationsWithinDistance(max) showLocationUnavailableForFilter() showInvalidMaxDistance() showDistanceFilterActive(max)** | Dan la ubicación del usuario y muestran la lista con las distancias, el mensaje sin resultados, el aviso de ubicación no disponible, el error del campo y la distancia activa en el subtítulo. | Interfaz (UI.1, UI.2) |

## Pruebas unitarias

### Pruebas unitarias de la clase StationFilter

Se aplican técnicas de clases de equivalencia y AVL para la definición de los casos de prueba. Además, se deberá garantizar la cobertura de sentencias y de condición/decisión. El usuario está en (43.4623, -3.8099). Salvo que se indique otra cosa, la lista de partida, que llamaremos L, contiene cinco estaciones situadas al norte del usuario, en este orden: A (a 0,289 km), B (0,801 km), C (0,9997 km, justo por debajo de 1 km), D (1,501 km) y E (sin coordenadas). Implementadas en StationFilterTest (directorio test, paquete es.unican.bicis.model).

### Método filterByMaxDistance(stations, userLat, userLon, maxDistKm)

**Tabla 3. Casos de prueba del método filterByMaxDistance**

| Identificador | Entrada | Valor esperado |
|---|---|---|
| UFD.1a | L, maxDistKm = 1.0 (Caso de éxito) | [A, B, C, E] |
| UFD.1b | L, maxDistKm = 0.5 | [A, E] |
| UFD.1c | L, maxDistKm = 2.0 (Todas dentro) | [A, B, C, D, E] (E al final) |
| UFD.1d | L, maxDistKm = 0.05 (Ninguna dentro) | [] (tampoco E) |
| UFD.1e | stations = null, maxDistKm = 1.0 | [], sin excepción |

| Identificador | Entrada | Valor esperado |
|---|---|---|
| UFD.1f | L, maxDistKm = 0.0 (Límite no válido) | [] |
| UFD.1g | L, maxDistKm = -1.0 (Distancia negativa) | [] |
| UFD.1h | stations = [], maxDistKm = 1.0 | [] |
| UFD.1i | stations = [E], maxDistKm = 1.0 (Solo estaciones sin coordenadas) | [] |
| UFD.1j | L, maxDistKm = distanceKm(usuario, C) (Igual a la distancia de C: valor límite) | [A, B, C, E] |
| UFD.1k | L, maxDistKm = distanceKm(usuario, C) - 0.001 (Justo por debajo: valor límite) | [A, B, E] |
| UFD.1l | stations = [C, A], maxDistKm = 1.0 | [C, A] (se mantiene el orden) y la lista no se modifica |

### Método hasValidCoordinates(station)

**Tabla 4. Casos de prueba del método hasValidCoordinates**

| Identificador | Entrada | Valor esperado |
|---|---|---|
| UHC.1a | lat = 43.46, lon = -3.81 | true |
| UHC.1b | station = null | false |
| UHC.1c | lat = null, lon = -3.81 | false |
| UHC.1d | lat = 43.46, lon = null | false |
| UHC.1e | lat = 90.0, lon = 180.0 (valores límite máximos) | true |
| UHC.1f | lat = -90.0, lon = -180.0 (valores límite mínimos) | true |
| UHC.1g | lat = 91.0, lon = -3.81 (latitud fuera de rango) | false |
| UHC.1h | lat = 43.46, lon = 181.0 (longitud fuera de rango) | false |

### Método distanceKm(lat1, lon1, lat2, lon2)

**Tabla 5. Casos de prueba del método distanceKm**

| Identificador | Entrada | Valor esperado |
|---|---|---|
| UDK.1a | Mismo punto: (43.4623, -3.8099) y (43.4623, -3.8099) | 0.0 |
| UDK.1b | Usuario (43.4623, -3.8099) y A (43.4649, -3.8099) | 0.289 km (con una tolerancia de 0.001) |
| UDK.1c | Los mismos puntos en orden inverso: A y usuario | El mismo valor que UDK.1b (simetría) |

### Pruebas unitarias del presenter StationsPresenter

Se prueba el presenter aislado. La vista (IStationsContract.View) y el repositorio (INetworksRepository) se sustituyen por mocks de Mockito; el repositorio responde con las estaciones indicadas. La red es TUeBICI (Santander, ES). Salvo que se indique otra cosa, hay

cuatro estaciones cargadas: A (a 0,3 km, 0 bicis), B (0,8 km, 5 bicis), C (1,5 km, 3 bicis) y D (sin coordenadas, 2 bicis), y la vista devuelve getUserLocation() = {43.4623, -3.8099}. Implementadas en StationsPresenterTest (directorio test, paquete es.unican.bicis.activities.stations).

**Métodos onDistanceFilterApplied(maxDistKmText) y getMaxCityDistanceKm()**

**Tabla 6. Casos de prueba de los métodos del presenter**

| Identificador | Entrada | Valor esperado |
|---|---|---|
| UPD.1a | onDistanceFilterApplied("1.0") | showDistanceFilterActive(1.0) y showStationsWithDistance([A, B, D], distancias de A y B); cabecera "Santander, ES · 3 estaciones" |
| UPD.1b | " 1,0 " (con espacios y coma decimal) | Igual que UPD.1a |
| UPD.1c | "0.05" (ninguna dentro) | showNoStationsWithinDistance(0.05). No se llama a showStationsWithDistance |
| UPD.1d – 1h | "0", "-1", "abc", "", null (Valores no válidos) | showInvalidMaxDistance() y no se filtra ni cambia la lista |
| UPD.1i | "1.0" con getUserLocation() = null | showLocationUnavailableForFilter() y no se filtra |
| UPD.1j | "1.0" con el filtro de bicis activo (mínimo 1) | showStationsWithDistance([B, D]): A no tiene bicis (se cumplen los dos filtros) |
| UPD.1k | "2.0" con las estaciones cargadas en orden [C, B, A, D] | showStationsWithDistance([A, B, C, D]): de la más cercana a la más lejana, D al final |
| UPD.2a | getMaxCityDistanceKm() | 1,5 km (C, la estación más lejana con coordenadas) |
| UPD.2b | getMaxCityDistanceKm() con getUserLocation() = null | 0.0 |

## Reporte final

Resultado de la ejecución de las pruebas de la historia. Rellenar al terminar el Sprint.

**Tabla 7. Resultado de las pruebas**

| Pruebas | Autor / ejecutor | Resultado |
|---|---|---|
| Unitarias StationFilter (UFD.1, UHC.1, UDK.1) | Pablo Casal | Pendiente |
| Unitarias StationsPresenter (UPD.1, UPD.2) | Pablo Casal | Pendiente |
| Interfaz UI.1 – UI.2 | David González | Pendiente |
| Aceptación Pruebas 1 – 4 | Diego Alonso | Pendiente |

Fallos detectados gracias a las pruebas: (rellenar al terminar el Sprint).

## Anexo 1. Datos de prueba

Los datos con los que se lanza la aplicación en modo debug son los de res/raw/networks.json y res/raw/network_tuebici.json. TUeBICI tiene 42 estaciones. La ubicación del usuario se simula en

(43.4623057, -3.8099803), que son las coordenadas de la propia red TUeBICI (centro de Santander, junto al Ayuntamiento). Para poder probar el caso de una estación cuya distancia no se puede calcular se ha modificado una de ellas, como se muestra en la Tabla 8. Se ha elegido una estación que no se usa en las pruebas de otras historias. La Tabla 9 indica cuántas estaciones deben verse para cada distancia y la Tabla 10, la distancia de las estaciones que aparecen en las pruebas.

**Tabla 8. Estación de TUeBICI modificada para las pruebas**

| Estación | Dato original | Dato en los datos de prueba |
|---|---|---|
| Valdenoja 2 (NE) | latitude y longitude informadas (a 1,98 km del usuario) | Sin latitude ni longitude (no informa) |

**Tabla 9. Resultado esperado del filtro por distancia sobre TUeBICI**

| Distancia máxima | Estaciones mostradas | Primera y última con distancia | Se usa en |
|---|---|---|---|
| Sin filtro | 42 | – | – |
| 0,05 km | 0 (mensaje sin resultados) | – | Prueba 3, UI.2 |
| 0,437 km | 4 + Valdenoja 2 (NE) = 5 | Ayuntamiento (NE) … Renfe | Prueba 2 |
| 1 km | 12 + Valdenoja 2 (NE) = 13 | Ayuntamiento (NE) … Tabacalera/Varadero | Pruebas 1, 3 y 4, UI.1 |
| 3 km | 32 + Valdenoja 2 (NE) = 33 | Ayuntamiento (NE) … Magdalena | Prueba 4 |
| 5,07 km (máximo de la ciudad, valor por defecto) | 41 + Valdenoja 2 (NE) = 42 | Ayuntamiento (NE) … PTCAN 1 | – |

**Tabla 10. Distancia al usuario de las estaciones usadas en las pruebas**

| Estación | Distancia al usuario |
|---|---|
| Ayuntamiento (NE) | 0,081 km (la más cercana) |
| Renfe | 0,437 km |
| Tabacalera/Varadero | 0,967 km |
| Instituto Pereda (NE) | 1,062 km |
| Magdalena | 2,836 km |
| Alisal (NE) | 3,397 km |
| PTCAN 1 | 5,067 km (la más lejana) |
| Valdenoja 2 (NE) | No se puede calcular (sin coordenadas) |